Homework 2

Question 3.1

Using the same data set (credit_card_data.txt or credit_card_data-headers.txt) as in Question 2.2, use the ksvm or kknn function to find a good classifier:

3.1.a using cross-validation (do this for the k-nearest-neighbors model; SVM is optional)

We import the relevant packages and read the credit card dataset into a dataframe

```
# Importing packages
library(kernlab)
library(caret)
library(kknn)

# Read credit card data into df
directory <- "C:/Users/Clayton/Documents/Gatech/ISYE 6501 Intro to Analytics Modelling/Module 2
Classification/"
df <- read.table(paste0(directory, "credit_card_data-headers.txt"), header = T, sep = "\t")
```

Set seed for reproducible results

```
set.seed(31)
```

Since we are doing a k-fold cross validation, we will simply split the data to a training set (80%) and a test set (20%), where the training set will be used for cross validation and the test set will be used to validate the predictions from the model

```
# create train-test split (80-20)
df_len <- dim(df)[1]
test_sample <- sample(1:df_len, size=round(0.2*df_len), replace=FALSE)
train_set <- df[-test_sample,]
test_set <- df[test_sample,]
```

K-nearest-neighbours model

First, we will train the k-nearest-neighbours model using the training set. Setting the conditions of the cross-validation, the training control method is a 10-fold cross validation, with a hyperparameter tuning grid for kknn model for distance (in range 1 to 5), kmax (value of 20), and kernels (value of optimal).

```
# define training control for cross validation
train_control <- trainControl(method = "cv", number = 10)

# define Hyperparameter Grid
tune_grid <- expand.grid(distance=c(1:5), kmax=20, kernel="optimal")
```

We then train the model with cross-validation to identify the best performing hyperparameters from the tuning grid. In this case, the best hyperparameters are:

- kmax = 20
- k = 13
- distance = 1
- kernel = ‘optimal’

```
# train the kknn classification model with cross validation
model <- train(as.factor(R1)~., data=train_set, trControl=train_control, tuneGrid=tune_grid, met
hod="kknn")
```

```
## Best value of k: 13
```

```
## Best tuned distance hyperparameter: 1
```

Finally, we train the model again with the best hyperparameters and value of k to derive our model for predictions. With this model, we achieved an accuracy score of `84.73%`

```
# using the best parameters from the training step, build the kknn model
kknn_model <- kknn(as.factor(R1)~., train=train_set, test=test_set, scale=T, k=13, distance=1, k
ernel='optimal')

# fit the model and observe the model performance
kknn_fit <- fitted(kknn_model)
confusionMatrix(data=kknn_fit, reference=as.factor(test_set$R1))
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction  0  1
##          0 64 14
##          1  6 47
##
##                Accuracy : 0.8473
##                  95% CI : (0.7741, 0.9042)
##     No Information Rate : 0.5344
##     P-Value [Acc > NIR] : 3.496e-14
##
##                   Kappa : 0.6906
##
##  Mcnemar's Test P-Value : 0.1175
##
##             Sensitivity : 0.9143
##             Specificity : 0.7705
##          Pos Pred Value : 0.8205
##          Neg Pred Value : 0.8868
##              Prevalence : 0.5344
##          Detection Rate : 0.4885
##    Detection Prevalence : 0.5954
##       Balanced Accuracy : 0.8424
##
##        'Positive' Class : 0
##
```

KSVM model

Similarly, we will do the cross-validation on the training set using a 10-fold cross-validation.

```
# create folds for k-fold cross validation
folds = createFolds(as.factor(train_set[, 11]), k = 10)
```

After which, we will optimise for the best value of C for the Support Vector Model by assessing the 10-fold cross-validation model performance. Through this optimisation, we found the best value of C to be `0.005`, achieving an accuracy of `85.27%`.

```
## The best performing value of C is: 0.005
```

```
## The accuracy with the best value of C is: 85.27448 %
```

With the optimised value of C, we now train the model for validation against the test set. With this model, we achieved an accuracy score of `90.84%`.

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction  0  1
##          0 59  1
##          1 11 60
##
##                Accuracy : 0.9084
##                  95% CI : (0.8455, 0.9518)
##     No Information Rate : 0.5344
##     P-Value [Acc > NIR] : < 2.2e-16
##
##                   Kappa : 0.8178
##
##  Mcnemar's Test P-Value : 0.009375
##
##             Sensitivity : 0.8429
##             Specificity : 0.9836
##          Pos Pred Value : 0.9833
##          Neg Pred Value : 0.8451
##              Prevalence : 0.5344
##          Detection Rate : 0.4504
##    Detection Prevalence : 0.4580
##       Balanced Accuracy : 0.9132
##
##        'Positive' Class : 0
##
```

3.1.b splitting the data into training, validation, and test data sets (pick either KNN or SVM; the other is optional).

In this question, we will split the data into training (70%), validation (15%), and test (15%) sets. This is to ensure the majority of data points are allocated for training.

```
# create train-validation-test split (70-15-15)
set.seed(31)

df_len <- dim(df)[1]
test_sample <- sample(1:df_len, size=round(3/20*df_len), replace=FALSE)
train_validation_set <- df[-test_sample,]
test_set <- df[test_sample,]

train_validation_df_len <- dim(train_validation_set)[1]
validation_sample <- sample(1:train_validation_df_len, size=round(3/17*train_validation_df_len),
replace=FALSE)
train_set <- df[-validation_sample,]
validation_set <- df[validation_sample,]
```

KNN model

Similar to 3.1.a, we first train the model to find the best value of k. With this train step, we find the best value of k to be `16` which achieved an accuracy score of `85.97%`.

```
# train the kknn model to find the best value of k
kknn_model <- train.kknn(as.factor(R1)~., data=train_set, scale=T, kmax=50)
best_k <- kknn_model[['best.parameters']][[2]]
best_accuracy <- 1 - min(kknn_model[['MISCLASS']])
```

```
## The best performing value of k is: 16
```

```
## The accuracy with the best value of k is: 85.97122 %
```

With this model, we achieved an accuracy score of `76.53%` on the validation set.

```
# fit the model and observe the model performance
y_pred <- predict(kknn_model, validation_set[, 1:10])
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction  0  1
##          0 35 13
##          1 10 40
##
##                Accuracy : 0.7653
##                  95% CI : (0.6689, 0.845)
##     No Information Rate : 0.5408
##     P-Value [Acc > NIR] : 3.601e-06
##
##                   Kappa : 0.5298
##
##  Mcnemar's Test P-Value : 0.6767
##
##             Sensitivity : 0.7778
##             Specificity : 0.7547
##          Pos Pred Value : 0.7292
##          Neg Pred Value : 0.8000
##              Prevalence : 0.4592
##          Detection Rate : 0.3571
##    Detection Prevalence : 0.4898
##       Balanced Accuracy : 0.7662
##
##        'Positive' Class : 0
##
```

KSVM model

For the KSVM model, we first optimise for the value of C. In this step, we find the value of C to be `0.005` which achieved an accuracy score of `86.87%`.

```
## The best performing value of C is: 0.005
```

```
## The accuracy with the best value of C is: 86.8705 %
```

Using the best value of C from the previous step, we build the KSVM model for predictions on the validation set. This model achieved an accuracy score of `83.67%` on the validation set.

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction  0  1
##          0 32  3
##          1 13 50
##
##                Accuracy : 0.8367
##                  95% CI : (0.7484, 0.9037)
##     No Information Rate : 0.5408
##     P-Value [Acc > NIR] : 6.135e-10
##
##                   Kappa : 0.6657
##
##  Mcnemar's Test P-Value : 0.02445
##
##             Sensitivity : 0.7111
##             Specificity : 0.9434
##          Pos Pred Value : 0.9143
##          Neg Pred Value : 0.7937
##              Prevalence : 0.4592
##          Detection Rate : 0.3265
##    Detection Prevalence : 0.3571
##       Balanced Accuracy : 0.8273
##
##        'Positive' Class : 0
##
```

Chosen model - KSVM Model

Since the KSVM model performed better on the validation set, we will evaluate the test set using KSVM. Comparing the model performance on the validation vs test set, the accuracy scores are fairly similar to each other; therefore, we can be fairly confident that we can observe a similar accuracy result when using the model on a new dataset.

```
ksvm_pred <- predict(ksvm_model, test_set[, 1:10])
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction  0  1
##          0 43  0
##          1  9 46
##
##                Accuracy : 0.9082
##                  95% CI : (0.8328, 0.9571)
##     No Information Rate : 0.5306
##     P-Value [Acc > NIR] : 6.268e-16
##
##                   Kappa : 0.8177
##
##  Mcnemar's Test P-Value : 0.007661
##
##             Sensitivity : 0.8269
##             Specificity : 1.0000
##          Pos Pred Value : 1.0000
##          Neg Pred Value : 0.8364
##              Prevalence : 0.5306
##          Detection Rate : 0.4388
##    Detection Prevalence : 0.4388
##       Balanced Accuracy : 0.9135
##
##        'Positive' Class : 0
##
```

Question 4.1

Describe a situation or problem from your job, everyday life, current events, etc., for which a clustering model would be appropriate. List some (up to 5) predictors that you might use.

A: In my job in the food delivery industry, we have restaurant accounts on our platform to provide food options to our customers. One use case of clustering for these restaurant vendors is to cluster them based on their performance on our platform so that we can target them more effectively with benefits to encourage their good performance or recommendations to improve their performance.

Some predictors we can use are:

- Number of years the vendor is on our platform
- Amount of Gross Merchandise Value contributed by the vendor in the past 3 months
- Number of orders by the vendor in the past 3 months
- Conversion rate (# orders / # clicks) of the vendor in the past 3 months
- Average customer rating of the vendor in the past 3 months

Question 4.2

The iris data set iris.txt contains 150 data points, each with four predictor variables and one categorical response. The predictors are the width and length of the sepal and petal of flowers and the response is the type of flower. The data is available from the R library datasets and can be accessed with iris once the library is loaded. It is also available at the UCI Machine Learning Repository (https://archive.ics.uci.edu/ml/datasets/Iris (https://archive.ics.uci.edu/ml/datasets/Iris)). The response values are only given to see how well a specific method performed and should not be used to build the model. Use the R function kmeans to cluster the points as well as possible. Report the best combination of predictors, your suggested value of k, and how well your best clustering predicts flower type.

First, we import the relevant libraries for this question.

```
library(datasets)
library(cluster)
library(factoextra)
```

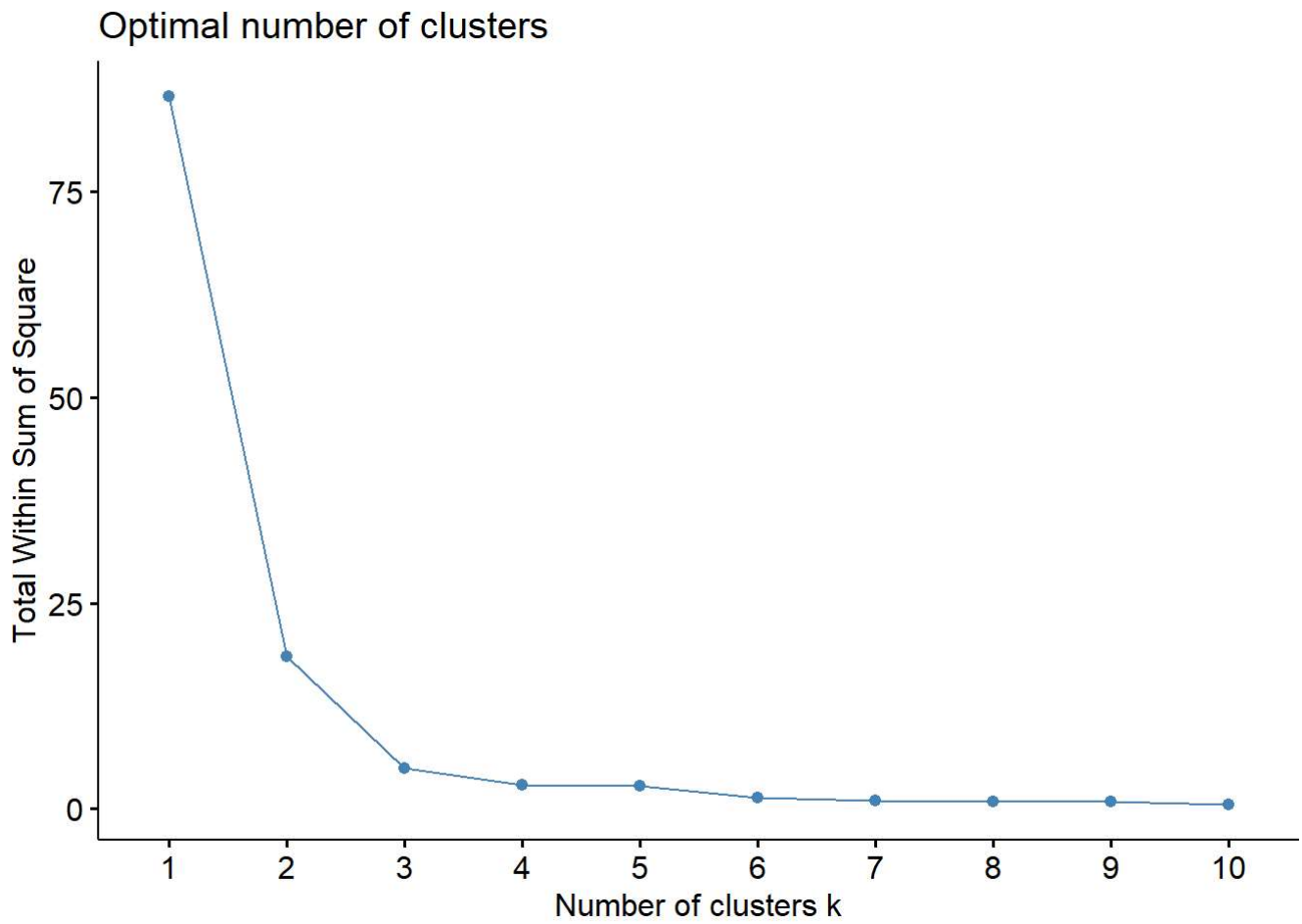
Next, we set a seed for the session for reproducibility.

```
set.seed(31)
```

Observing the iris dataset, we see that there are 3 categories for the target variable `Species` . This indicates that we should expect to train our k-means model for 3 clusters.

```
##   Sepal.Length    Sepal.Width     Petal.Length    Petal.Width   
##  Min.   :4.300   Min.   :2.000   Min.   :1.000   Min.   :0.100  
##  1st Qu.:5.100   1st Qu.:2.800   1st Qu.:1.600   1st Qu.:0.300  
##  Median :5.800   Median :3.000   Median :4.350   Median :1.300  
##  Mean   :5.843   Mean   :3.057   Mean   :3.758   Mean   :1.199  
##  3rd Qu.:6.400   3rd Qu.:3.300   3rd Qu.:5.100   3rd Qu.:1.800  
##  Max.   :7.900   Max.   :4.400   Max.   :6.900   Max.   :2.500  
##        Species  
##  setosa    :50  
##  versicolor:50  
##  virginica :50  
##                 
##                 
##                 
```

Despite identifying the 3 possible categories of Species, we can still plot an elbow plot to observe the likely number of optimal clusters for this dataset. From the plot, we see that the ‘elbow’ is at where `k = 3` , therefore, our chosen value of k is `3` .

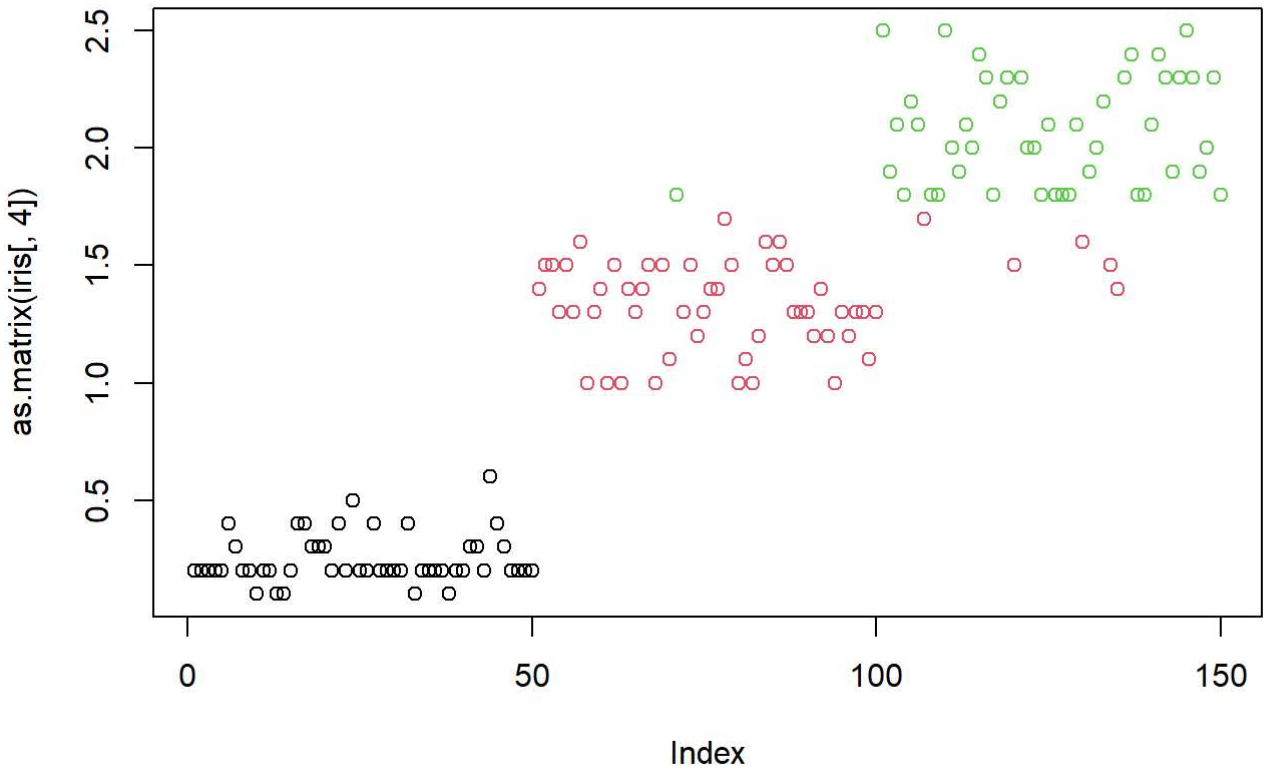


We can now train the model to identify 3 clusters by specifying the number of cluster centers. We are also doing a sort on the cluster centers to ensure reproducibility of the cluster numbers for each cluster.

From the different permutations of predictors, we find that using all 4 predictors for the model maximises the model performance when comparing predictions against its true values. Hence, we will include all 4 predictors in this model.

```
cluster_model <- kmeans(as.matrix(iris[, 4]), centers=3)
centers <- sort(cluster_model$centers)
cluster_model <- kmeans(as.matrix(iris[, 4]), centers=centers)
```

We can also plot the data points on a chart to observe how each point is clustered.



Due to the nature of the model assigning clusters to numbers, we are now renaming clusters to the respective species category.

```
y_pred <- factor(cluster_model$cluster)
levels(y_pred) <- c("setosa","versicolor","virginica")
```

By comparing the clustering results to the actual species category in the dataset, we can calculate the accuracy of this model in clustering each flower; this model has achieved an accuracy score of `96%` .

```
## Confusion Matrix and Statistics
##
##             Reference
## Prediction   setosa versicolor virginica
##   setosa         50          0         0
##   versicolor      0         49         5
##   virginica       0          1        45
##
## Overall Statistics
##
##                Accuracy : 0.96
##                  95% CI : (0.915, 0.9852)
##     No Information Rate : 0.3333
##     P-Value [Acc > NIR] : < 2.2e-16
##
##                   Kappa : 0.94
##
##  Mcnemar's Test P-Value : NA
##
## Statistics by Class:
##
##                      Class: setosa Class: versicolor Class: virginica
## Sensitivity                 1.0000            0.9800           0.9000
## Specificity                 1.0000            0.9500           0.9900
## Pos Pred Value              1.0000            0.9074           0.9783
## Neg Pred Value              1.0000            0.9896           0.9519
## Prevalence                  0.3333            0.3333           0.3333
## Detection Rate              0.3333            0.3267           0.3000
## Detection Prevalence        0.3333            0.3600           0.3067
## Balanced Accuracy           1.0000            0.9650           0.9450
```